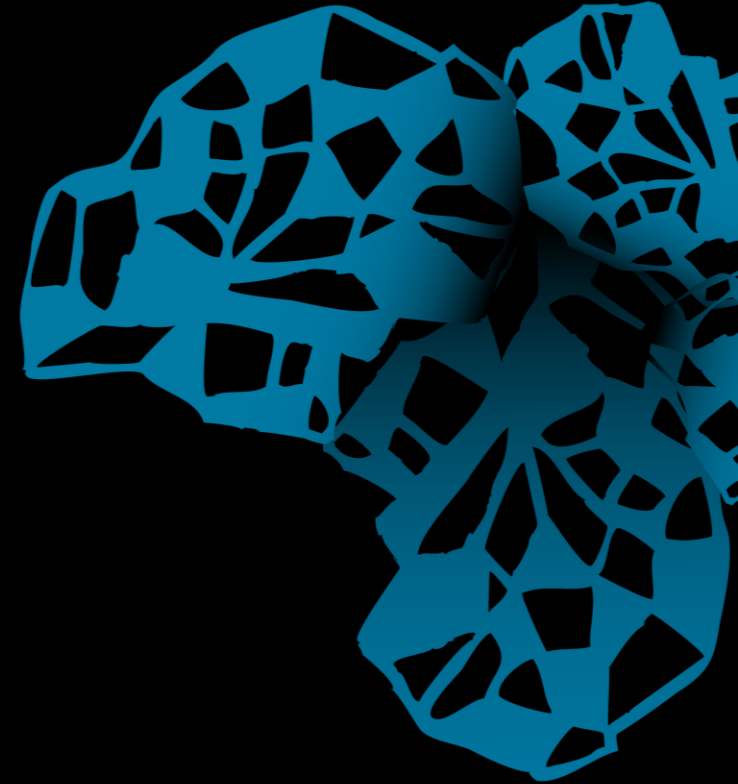


OPENINTEL/SPARK



Open INTEL in numbers:

308
MILLION

domains measured on a
daily basis

5.0
BILLION

data points collected daily

10.7
TRILLION

data points collected since
the start in 2015



OPENINTEL IS A JOINT PROJECT

INTRO

- *OpenINTEL dataset*
- *Introduction to PySpark*



HOW FAMILIAR ARE YOU WITH DNS?

```
$ dig @1.1.1.1 +dnssec A www.utwente.nl

; <<>> DiG 9.10.6 <<>> @1.1.1.1 +dnssec A www.utwente.nl
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 53758
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1

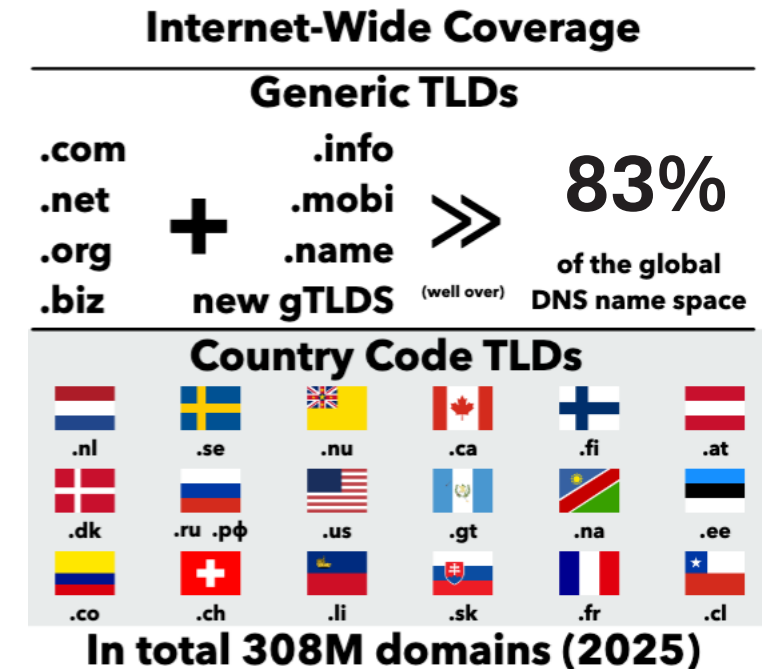
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
;; QUESTION SECTION:
;www.utwente.nl.                IN      A

;; ANSWER SECTION:
www.utwente.nl.                300     IN      CNAME   webhare.utwente.nl.
www.utwente.nl.                300     IN      RRSIG   CNAME 8 3 300 20260315204534 20260213200801 12285 utwente.nl. Rf5yjZx00XanKtBx
57ncMxxaI8IftCV8q4Cl+IR2T8prHQ67thysTgzh QBzsAZF97NUQhaEaCtiz6VRd15zJ3+qticiTEwhx7nHdixqg7q6JBzmF nSLz6Yr+rTTqAHGHYQZ2kxflqqz5
koV7ui3H7WFkDbptIa1AlQ7UeJdg kgc=
webhare.utwente.nl.           600     IN      A        130.89.12.12
webhare.utwente.nl.           600     IN      RRSIG   A 8 3 600 20260310122157 20260208112446 12285 utwente.nl. ck5YHP/p83PYsCCNFWZ+
LpMlznz403Ai74SIcCIiYc/PXAY4WGH5B69r uZhQwTPrPL+FvrcnDs6/Q5cgbkzsIx+yXWg1iJ2HIscfyMUa4YKK1LuR L044MNDr0BJuHFkSvcWRNPLn5bz8J20t
77K1y0Iu79YM5P8vtE7aK15v QPo=

;; Query time: 41 msec
;; SERVER: 1.1.1.1#53(1.1.1.1)
;; WHEN: Mon Feb 16 15:08:02 CET 2026
;; MSG SIZE rcvd: 421
```

OPENINTEL MEASUREMENT

- **Daily measurement of global DNS space since 2015**
- For the **apex of a domain name** we measure: SOA, NS, A, AAAA, MX, TXT, DNSKEY, DS, NSEC3, CAA, CDS, DNSKEY.
 - Selected queries for labels, e.g.: A/AAAA for **www..example.com**
- For the full IPv4 space we measure: PTR (reverse DNS)
- Measurements are seeded by:
 - Zone files (.com, .org, .nl, .ch, etc)
 - Top lists (Tranco, Radar, Umbrella, etc)
 - CT-Logs (Google Xenon, Cloudflare Nimbus, Let's Encrypt Oak, etc)
- Follow up **infrastructure measurement** for NS, MX and TLSA (DANE) names extracted from the above OpenINTEL measurements.
- Full details on <https://openintel.nl/>



query name	query type	response name	response type	ip4 address	cname name	mx address	txt text
utwente.nl.	A	utwente.nl.	A	130.89.12.12			
utwente.nl.	A	utwente.nl.	RRSIG				
utwente.nl.	AAAA	utwente.nl.	AAAA				
utwente.nl.	CAA	utwente.nl.	CAA				
utwente.nl.	DNSKEY	utwente.nl.	DNSKEY				
utwente.nl.	DS	utwente.nl.	DS				
utwente.nl.	MX	utwente.nl.	MX			utwente-nl.mail.protection.outlo...	
utwente.nl.	NS	utwente.nl.	NS				
utwente.nl.	NSEC3PARAM	utwente.nl.	NSEC3PARAM				
utwente.nl.	SOA	utwente.nl.	SOA				
utwente.nl.	TXT	utwente.nl.	TXT				"zoom=7028454"
utwente.nl.	TXT	utwente.nl.	TXT				"MS=ms81740447"
utwente.nl.	TXT	utwente.nl.	TXT				
utwente.nl.	TXT	utwente.nl.	TXT				"Wr3KIzpc3tUeVW9RvqJ5Qj....."
utwente.nl.	TXT	utwente.nl.	TXT				"atlassian-domain-verification...."
utwente.nl.	TXT	utwente.nl.	TXT				"v=spf1 mx a:smtppool.utwent...."
123-nonexistant-dnsjedi-456.ut...	AFSDB	utwente.nl.	SOA				
123-nonexistant-dnsjedi-456.ut...	AFSDB	9A9C5F608IAJ....	NSEC3				
_dmarc.utwente.nl.	TXT	_dmarc.utwente.nl.	TXT				"v=DMARC1; p=reject; sp=no...."
www.utwente.nl.	A	webhare.utwent...	A	130.89.12.12			
www.utwente.nl.	A	www.utwente.nl.	CNAME		webhare.utwente.nl.		
www.utwente.nl.	AAAA	webhare.utwent...	AAAA				
www.utwente.nl.	AAAA	www.utwente.nl.	CNAME		webhare.utwente.nl.		

Not all rows visible

9/98 columns visible – Columns not relevant to the query will be NULL

OPENINTEL DATA SCHEMA

Sparse storage – Nullable fields, populated based on response type of the DNS response)

Column	Type	Description
query_name	STRING	Domain queried
query_type	STRING	Query type (A, AAAA, MX...)
response_type	STRING	Response record type
response_name	STRING	Response domain name
ip4_address	STRING	IPv4 for A records
ip6_address	STRING	IPv6 for AAAA records

Column	Type	Description
mx_address	STRING	Mail server name
ns_address	STRING	Nameserver name
soa_mname	STRING	SOA master name
txt_text	STRING	TXT record content
timestamp	LONG	Measurement epoch (ms)
country	STRING	GeoIP country code

Full dictionary available here: <https://openintel.nl/data/dictionary/>

PySpark Code

```
# Explore schema  
df.printSchema()
```

```
# Select specific columns for efficiency  
df.select("query_name", "response_type", "ip4_address", "country").show(10)
```

 *Tip: We use Apache Parquet, which uses columnar storage. Selecting the columns first will reduce overall read time!*

ARE YOU FAMILIAR WITH DATAFRAME TOOLS?

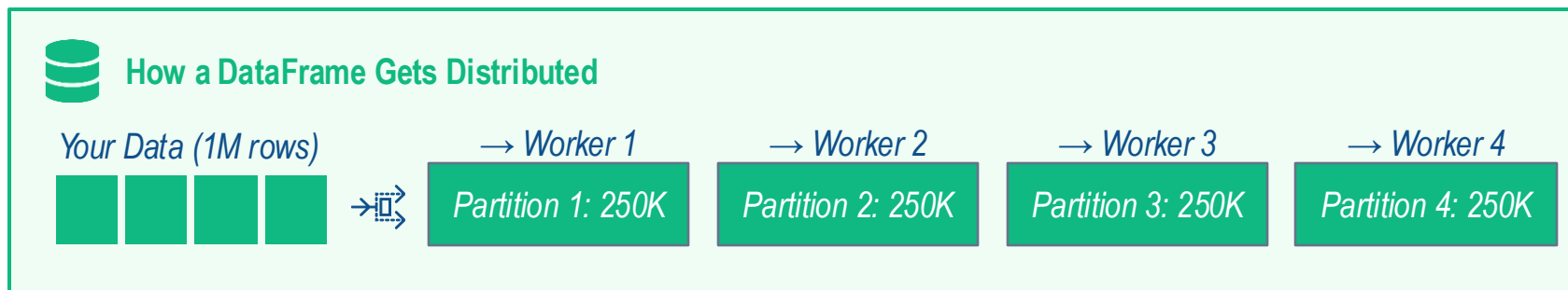
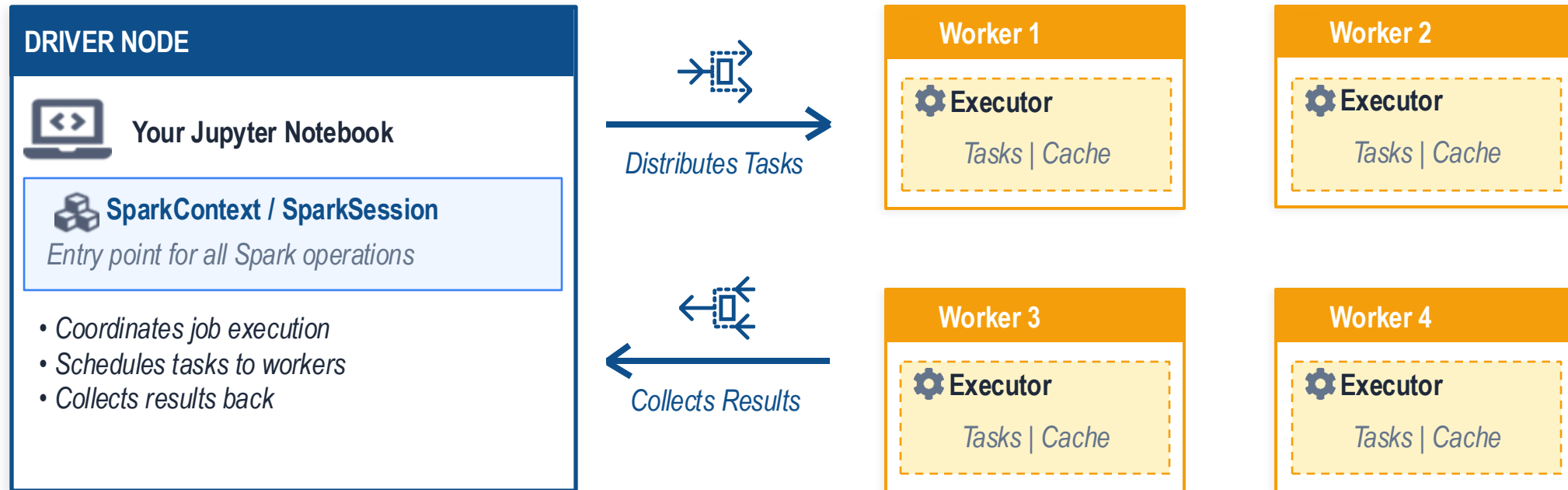


(PY)SPARK



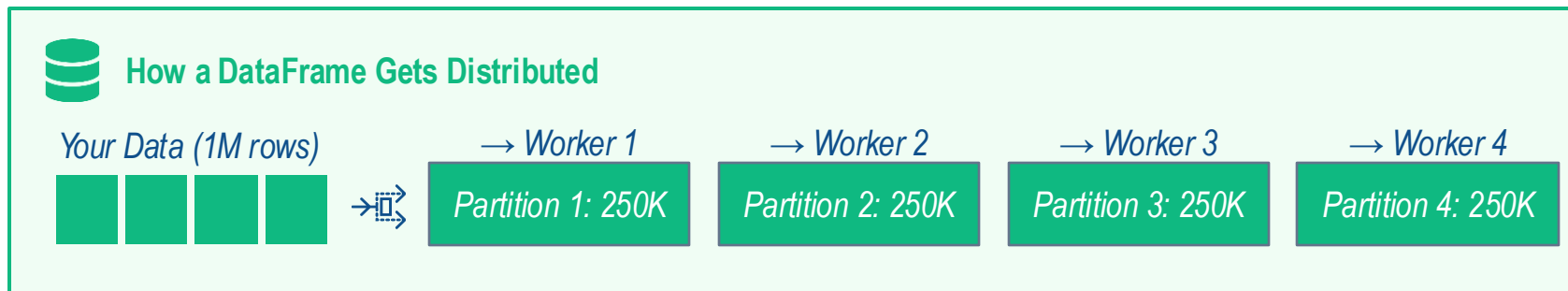
- *Query, Transform and Aggregate Data using Data Frames on a* **Distributed Cluster**

SPARK – DISTRIBUTED COMPUTING



SPARK – SOME TERMINOLOGY (1/2)

- *In Spark, we make the difference between **narrow** and **wide** transformations.*
- **Narrow transformations** *are transformations that depend on a **single** input partition.*
 - *(A transformation on a **single row** of the data frame)*
- **Wide transformations** *are transformations that depend on **multiple** input partitions.*
 - *(A transformation on **multiple rows** of the data frame)*



SPARK – SOME TERMINOLOGY (2/2)

- *In spark, **jobs**, **stages** and **tasks** describe how your code gets executed.*
- *A **job** is created whenever you call an **action** (e.g. `.show()`) – each action results in one job.*
- *A **stage** is a **set of tasks** that can be executed **without shuffling** data between executor nodes.*
- *A **task** is the smallest unit of work in Spark – represents the execution of a **single stage on a single partition**.*
- *Spark is **lazy** – execution of tasks and stages only happen in the context of a job.*
 - *‘Executing’ code in your notebook that does not include an action **will not trigger any Spark computations.****
- ***Wide transformations** require **multiple partitions**. Spark may need to **exchange data between executors**.*
 - *This is known as a **shuffle**.*
 - *Shuffles happen at the end of a stage, any following computations creates a **new stage**.*

LOADING DATA

```
# The public location of the Tranco fDNS dataset.
TRANCO = "s3a://openintel-public/fdns/basis=toplist/source=tranco"

# The data for 2026-01-01
TRANCO_20260101 = TRANCO + "/year=2026/month=01/day=01"

# Read from a path
tranco_df = spark.read.parquet(TRANCO_20260101)

# Read partitioned data
tranco_df = spark.read.option(
    "basePath", TRANCO
).parquet(
    TRANCO_20260101
)

# Call an action
tranco_df.show()
```

- Spark can read anything from basic **text** (csv/json/xml), **ORC**, **avro**, **parquet**, **JDBC connectors**, and more.
- Can read **partitioned** data.
- Workers read directly from storage
- Common **Actions** include **.show()**, **.count()**, **.write()**, **.collect()**, **.toPandas()**, **.printSchema()**
- **Lazy evaluation** - nothing runs until an **Action**.

WARNING!

NEVER use **.collect()**, **.toPandas()** or equivalent on large frames.

This **WILL** cause your notebook to go **out of memory**.

Do use these functions on **aggregated data** – e.g. creating plots

BASIC (NARROW) TRANSFORMATIONS

```
import pyspark.sql.functions as psf

# Filter by record type
df.filter(
    psf.col("response_type") == "A"
).show(10)

# Multiple conditions
df.filter(
    (psf.col("response_type") == "A") & (psf.col("country") == "NL")
).show(10)

# Select with alias
df.select(
    psf.col("query_name").alias("domain"),
    psf.col("country")
).show(10)

# Add computed columns
df.withColumn(
    "tld", psf.element_at(psf.split(psf.col("query_name"), "\\."), -2)
).show(10)
```

Key Takeaways

- **Transformations** *that depend on a single input partition.*
- **Common Narrow Transformations** include **.filter()**, **.map()**, **.select()**, **.sample()**
- **Chained narrow transformation** *can be pipelined in a single stage.*

WIDE TRANSFORMATIONS - AGGREGATIONS

```
# Counting the number of TXT records for all domains
df.select(
    "query_name",
    "txt_text"
).filter(
    psf.col("txt_text").isNotNull()
).groupBy(
    "query_name"
).count()
```

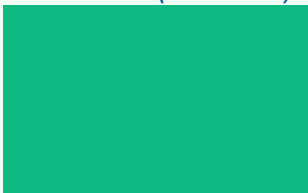
Key Takeaways

- Transformation that requires **multiple input partitions** that might live on **another executor**.
- Data moving between executors is called a **Shuffle**.
- Creates a **stage boundary**. Subsequent transformations (wide and narrow) will be executed in a new stage.
- Common **wide** transformations include `.groupBy()`, `.orderBy()`, `distinct()`, `.join()`



How a DataFrame Gets Distributed

Your Data (1M rows)



→ Worker 1

query_name	txt_text
utwente.nl	"zoom=7028454"
utwente.nl	"MS=ms81740447"

→ Worker 2

query_name	txt_text
utwente.nl	"apple-domain-ver..."
utwente.nl	"v=spf1 mx a:smtp..."

WIDE TRANSFORMATIONS - JOINS

```
# Wide Transformations – Joins
```

```
# Example: Join A records with MX records for same domain
```

```
a_records = tranco_df.filter(  
    (psf.col("response_type") == "A") &  
    (psf.col("ip4_address").isNotNull())  
).select("query_name").distinct()
```

```
mx_records = tranco_df.filter(  
    (psf.col("response_type") == "MX") &  
    (psf.col("mx_address").isNotNull())  
).select("query_name").distinct()
```

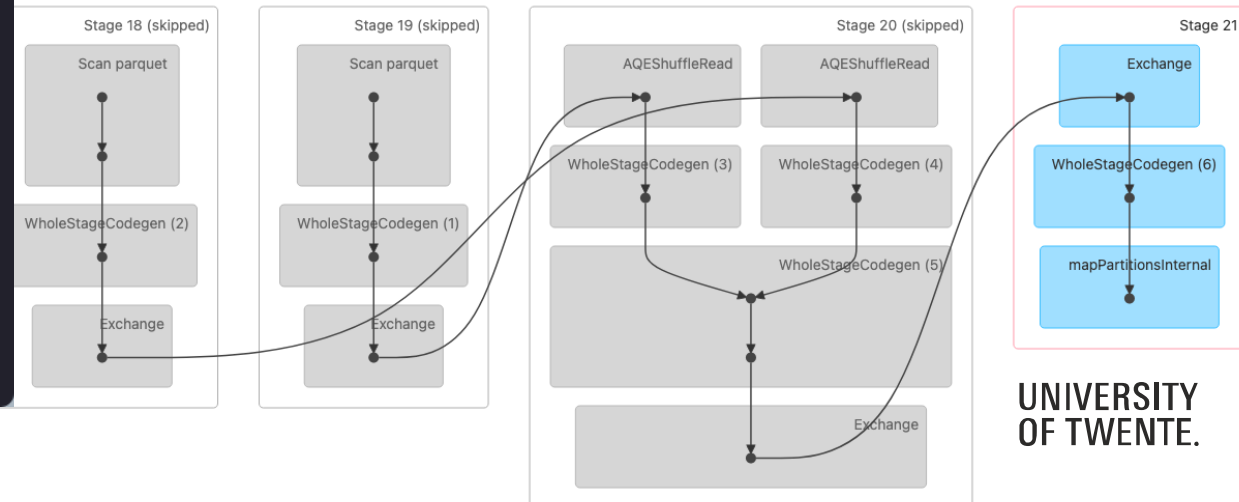
```
# Number of domains with both web and mail servers
```

```
combined = a_records.join(  
    mx_records,  
    on="query_name",  
    how="inner" # or "left", "right", "outer"  
).count()
```

Key Takeaways

Spark analyses your code and guesses which join strategy is best:

1. Broadcast Hash Join
 2. Shuffle Hash Join
 3. Sort-Merge Join
 4. Cartesian Join
- Same source data frame, but the data is **loaded from S3 twice**.



PERFORMANCE OPTIMIZATION

Column Pruning

Select required columns early

df.select('col1', 'col2')

Filter Pushdown

Filter before expensive operations

(Sometimes possible to data source driver)

df.filter(...).groupBy()

Caching

Cache frequently accessed data or intermediate data frames

df.persist()

Broadcast and Partitioning Hints

Broadcast small data frames before joining

df.broadcast() -> df2.join(df)

```
# See logical & physical execution plans
df.filter(
    psf.col("response_type") == "A"
).groupBy(
    "country"
).count().explain()

# Look for:
# - FileScan pushdown filters
# - Exchange (shuffle) operations
# - Join types: BroadcastHashJoin, SortMergeJoin, etc
```

Key Takeaways

- Show execution plan: **df.explain()**
- Broadcast vs shuffle joins
- Repartitioning strategies
- Caching strategies
- Monitoring with Spark UI

USER-DEFINED FUNCTIONS (UDF)

```
import pyspark.sql.types as pst
from publicsuffixlist import PublicSuffixList

# Initialize the PSL
psl = PublicSuffixList(only_icann=True)

@psf.udf(returnType=pst.StringType())
def extract_public_suffix(fqdn):
    """
    Extract the public suffix
    (e.g., 'co.uk' from 'mail.google.co.uk')
    using the publicsuffixlist library.
    """
    if fqdn is None:
        return None

    # Get the suffix
    suffix = psl.publicsuffix(fqdn)

    return suffix
```

```
# Get the top Public Suffixes and their counts
tranco_df.filter(
    psf.col("response_type") == "SOA"
).select(
    "query_name"
).withColumn(
    "tld", extract_public_suffix(psf.col("query_name"))
).groupBy(
    "tld"
).count().orderBy(
    "count", ascending=False
).show()
```

Key Takeaways

- You can run **your own python code** on data frames
- Library needs to be present on the executors!
- **MUCH slower** than native Spark functions

SAVING DATA

Key Takeaways

- *Executors have **no local storage**.*
 - *You won't be able to write to a local path*
- *Use Parquet for intermediate results (compression + speed)*
- ***.coalesce(n)** reduces partitions without shuffle*
- ***.repartition(n)** for better parallelism on large outputs*

 *Never call **toPandas()** on large Data Frames*

It loads everything into driver memory!

 *Delete your intermediate stored results after your research is completed!*

```
# Single partition to S3
result_df.write.parquet(
    "s3://my-bucket/path/analysis.parquet"
)

# Partitioned output with compression
result_df.write.option(
    "compression", "gzip"
).partitionBy(
    "year", "month"
).parquet(
    "s3://my-bucket/path/output/"
)

# Overwrite existing
result_df.write.mode(
    "overwrite"
).parquet(
    "s3://my-bucket/path/output/"
)

# Single CSV file
result_df.coalesce(1).write.csv(
    "s3://my-bucket/path/output/",
    header=True
)

# To Pandas (small data only!)
pdf = result_df.toPandas()
```

OPENINTEL DATASET – PARTICULARITIES 1/2

- *The datasets contain more than just DNS queries for the apex, we also query*
 - *A/AAAA for **www.example.com***
 - *TXT for the **_dmarc.example.com***
 - *AFSDB for **123-nonexistent-dnsjedi-456.example.com** (To trigger DNSSEC NSEC(3) responses)*
 - *Full lists of queries can be found on <https://openintel.nl> -> Data*
- *The measurement follows **CNAMEs** and stores them under the **original query_name***
 - *Check the **response_name** instead.*
- *A **single TXT** record may be composed of **multiple strings** of max 255 characters (octets) each.*
 - *We store all strings in the **txt_text** column, where each string is wrapped in quotation marks: “”.*
 - *You need to concatenate these strings (without spaces) yourself. ([RFC7208 Sec. 3.3](#))*

OPENINTEL DATASET – PARTICULARITIES 2/2

- The **status_code** column contains the values for DNS [as defined by IANA](#).
 - Common values include: 0=NoError, 2=ServFail, 3=NXDomain
 - **OpenINTEL meta value** for timeouts: 65533
- Domains might have **incomplete data**, the **measurement aborts** when encountering a **ServFail** or **Timeout**.
 - Both are stored with **NULL response_name** and **response_type**.
- The OpenINTEL schema has evolved over the years.
 - E.g.: 'spf_text' column still present – no longer populated. Check TXT at apex instead.

OpenINTEL is an Internet measurement.
Expect to encounter the unexpected.